

DOVA: When Agents Learn to Think Together

Building an Agentic Fabric for Deep Research Automation

DOVA Team

\begin{center} \Large\textit{"The whole is greater than the sum of its parts."} — Aristotle \end{center}

\vspace{1em}

The Problem with Single-Agent Systems

We've all experienced it. You ask an AI assistant to research a complex topic, and it returns a surface-level summary that misses nuance, ignores contradictions, and lacks the depth you'd expect from a human expert. The problem isn't intelligence—it's architecture.

Single-agent systems operate in isolation. They think linearly, conclude prematurely, and cannot benefit from the cognitive diversity that makes human research teams effective. When a lone researcher tackles a complex problem, they bring one perspective. When a team collaborates—each member with different expertise, each challenging the others' assumptions—something remarkable emerges: **collective intelligence**.

DOVA (Deep Orchestrated Versatile Agent) is our answer to this challenge. It's not just another AI research tool. It's an *agentic fabric*—a living system where specialized agents think independently, collaborate deliberately, and reason together to produce insights no single agent could achieve alone.

The Agentic Fabric: A New Mental Model

Traditional multi-agent systems treat agents as workers on an assembly line: one agent fetches data, another summarizes, a third formats output. This is parallelism, not collaboration.

DOVA introduces something different. We call it the **Agentic Fabric**—a dynamic mesh where agents don't just divide work, they *multiply insight*. Each agent brings specialized capabilities:

Agent	Role	Superpower
Research	Knowledge acquisition	Deep source traversal across ArXiv, GitHub, HuggingFace
Profiling	User understanding	Learns your interests, adapts recommendations
Validation	Truth verification	Executes code in sandboxes, tests claims
Synthesis	Insight integration	Weaves threads into coherent narratives
Bull/Bear	Dialectic reasoning	Adversarial debate surfaces blind spots

But the magic isn't in the agents themselves—it's in how they *reason together*.

Individual Intelligence: The ReAct Loop

Before agents can collaborate effectively, each must think well independently. DOVA agents use a **ReAct-style reasoning loop**: Think → Act → Observe → Reflect.

This isn't prompt engineering. It's cognitive architecture.

```
async def reason(
    self,
    problem: str,
    max_iterations: int = 5,
    reflect: bool = True,
) -> ReasoningTrace:
    """ReAct loop: Thought → Action → Observation → (repeat)"""
    trace = ReasoningTrace(problem=problem)
    self._scratchpad = {"observations": []}

    for iteration in range(max_iterations):
```

```

# THOUGHT: Reason about what to do next
thought = await self._think_step(problem, trace)
trace.steps.append(ReasoningStep(
    step_type=StepType.THOUGHT,
    content=thought["reasoning"],
    confidence=thought.get("confidence", 0.5),
))

# Check if ready to conclude
if thought.get("action") == "conclude":
    trace.final_answer = thought.get("conclusion", "")
    break

# ACTION: Execute chosen action
result = await self._action_step(thought["action"])

# OBSERVATION: Record what was learned
trace.steps.append(ReasoningStep(
    step_type=StepType.OBSERVATION,
    content=str(result),
))
self._scratchpad["observations"].append(result)

# REFLECTION: Self-critique before finalizing
if reflect and trace.final_answer:
    refined, critique = await self.reflect(trace.final_answer)
    trace.refined_answer = refined

return trace

```

Key insight: The `_scratchpad` acts as working memory. Unlike stateless LLM calls, agents accumulate context across reasoning steps. They remember what they've tried, what worked, and what didn't.

The reflection step is crucial. Before an agent commits to an answer, it asks: *"What did I miss? Where might I be wrong?"* This self-critique catches errors that would otherwise propagate through the system.

Collective Intelligence: Three Collaboration Patterns

Individual reasoning is necessary but not sufficient. DOVA implements three collaboration patterns that enable agents to achieve what we call **synergistic reasoning**—where 1+1 genuinely exceeds 2.

Pattern 1: The Blackboard

Imagine a shared whiteboard where researchers post hypotheses, evidence, and critiques. Each can see what others have written, build upon promising ideas, and challenge weak arguments.

```
class Blackboard:
    """Shared workspace for collaborative agent reasoning."""

    async def post(
        self,
        agent_name: str,
        post_type: PostType, # HYPOTHESIS, EVIDENCE, REFINEMENT
        content: str,
        confidence: float = 0.5,
        references: list[str] | None = None, # Posts this builds upon
    ) -> str:
        """Post an insight for others to see and build upon."""
        post = BlackboardPost(
            id=f"post_{uuid4().hex[:8]}",
            agent_name=agent_name,
            post_type=post_type,
            content=content,
            confidence=confidence,
            references=references or [],
        )
        self._posts[post.id] = post
        return post.id

    async def get_context(
        self,
        agent_name: str,
        exclude_own: bool = True,
        post_types: list[PostType] | None = None,
    ) -> list[BlackboardPost]:
        """Get relevant posts, excluding own contributions."""
        posts = [p for p in self._posts.values()
                  if not (exclude_own and p.agent_name == agent_name)]
        posts.sort(key=lambda p: p.weighted_confidence, reverse=True)
        return posts
```

Why it works: The Blackboard pattern enables *asynchronous collaboration*. Agents don't wait for each other. They contribute when ready, build on what's available, and let confidence-weighted voting surface the best ideas.

Pattern 2: The Ensemble

When the problem is well-defined but the solution space is vast, throw multiple minds at it simultaneously.

```

class EnsembleReasoning:
    """Multiple agents tackle the same problem in parallel."""

    async def reason(
        self,
        problem: str,
        agents: list[Any],
        method: AggregationMethod = AggregationMethod.SYNTHESIS,
    ) -> EnsembleResult:
        # Launch all agents in parallel
        tasks = [self._get_agent_answer(agent, problem)
                  for agent in agents]
        answers = await asyncio.gather(*tasks, return_exceptions=True)

        valid_answers = [a for a in answers if isinstance(a, AgentAnswer)]

        # Aggregate based on method
        if method == AggregationMethod.SYNTHESIS:
            return await self._aggregate_synthesis(valid_answers, problem)
        elif method == AggregationMethod.VOTE:
            return self._aggregate_vote(valid_answers)
        elif method == AggregationMethod.BEST_OF:
            return self._aggregate_best_of(valid_answers)

```

The synthesis aggregation is where magic happens. Rather than simple voting, an LLM examines all perspectives and crafts a unified answer that incorporates the strongest elements from each while noting disagreements.

Pattern 3: Iterative Refinement

Some problems need not parallel exploration but *serial depth*. One agent proposes, another critiques, a third refines.

```

async def _iterative_reasoning(
    self,
    problem: str,
    agents: list[Any],
    max_iterations: int,
) -> CollaborativeResult:
    # Start with first agent's answer
    trace = await agents[0].reason(problem)
    current_answer = trace.refined_answer or trace.final_answer

    history = [{"agent": agents[0].name, "answer": current_answer}]

    # Each subsequent agent refines
    for iteration in range(1, max_iterations):
        agent = agents[iteration % len(agents)]

        refined, critique = await agent.reflect(current_answer, problem)

```

```

        current_answer = refined
        history.append({
            "agent": agent.name,
            "critique": critique,
            "refined": refined,
        })

    return CollaborativeResult(
        final_answer=current_answer,
        refinement_history=history,
    )

```

The power of critique: Each refinement pass doesn't just polish—it *stress-tests*. Weak arguments get strengthened or discarded. Gaps get filled. The final output has survived multiple rounds of intelligent scrutiny.

The Hybrid Mode: Orchestrating Orchestration

Real research problems don't fit neatly into one pattern. DOVA's **Hybrid Mode** chains all three:

$\text{Ensemble} \rightarrow \text{Blackboard} \rightarrow \text{Iterative Refinement}$

```

async def _hybrid_reasoning(self, problem: str, agents: list[Any]):
    # Step 1: Ensemble for diverse initial perspectives
    ensemble_result = await self._ensemble_reasoning(problem, agents)

    # Step 2: Post to blackboard, gather evidence
    self.blackboard.clear()
    await self.blackboard.post(
        agent_name="ensemble",
        post_type=PostType.HYPOTHESIS,
        content=ensemble_result.final_answer,
        confidence=ensemble_result.confidence,
    )

    # Add dissenting views as counter-evidence
    for view in ensemble_result.dissenting_views:
        await self.blackboard.post(
            agent_name="dissent",
            post_type=PostType.EVIDENCE,
            content=view,
            confidence=0.3,

```

```
)

# Step 3: Iterative refinement on the synthesized answer
final = await self._iterative_reasoning(
    problem=f"Refine: {ensemble_result.final_answer}",
    agents=agents[:2],
    max_iterations=2,
)

return CollaborativeResult(
    final_answer=final.final_answer,
    confidence=(ensemble_result.confidence + final.confidence) / 2,
    mode_used=CollaborationMode.HYBRID,
)
```

This isn't arbitrary chaining. Each stage serves a purpose:

1. **Ensemble** explores the solution space broadly
2. **Blackboard** preserves minority viewpoints and gathers supporting evidence
3. **Iterative refinement** deepens and polishes the consensus

Why This Matters: The Value Proposition

For Researchers

Stop context-switching between ArXiv, GitHub, and documentation. DOVA's agents traverse sources in parallel, cross-reference findings, and surface connections you might miss.

For Teams

Research insights are captured with full reasoning traces. When an agent concludes something, you can audit *why*—every thought, action, and reflection is logged.

For Production Systems

Configurable reasoning depth lets you balance thoroughness against latency:

Mode	Use Case	Behavior
quick	Real-time queries	Single-pass, no reflection

Mode	Use Case	Behavior
standard	Most requests	ReAct + self-reflection
deep	Complex analysis	Full ensemble reasoning
collaborative	Research projects	Hybrid multi-pattern

Key Takeaways

What We Learned Building DOVA

1. Reasoning > Prompting.} Structured cognitive loops (ReAct, reflection) consistently outperform clever prompts. Give agents architecture, not instructions.

2. Collaboration patterns matter.} Blackboard for async contribution, Ensemble for parallel exploration, Iterative for serial depth. Match the pattern to the problem.

3. Working memory is essential.} The scratchpad transforms stateless LLM calls into stateful reasoning. Agents that remember what they've tried make better decisions.

4. Dissent is data.} Don't discard minority opinions—surface them. The ensemble's disagreements often point to genuine complexity in the problem.

5. Audit trails enable trust.} Full reasoning traces let users (and developers) understand not just what the system concluded, but how and why.

Getting Started

```
# Install DOVA
pip install -e ".[dev]"

# Run a collaborative research query
```



```
dova research "compare RAG vs fine-tuning for domain adaptation" \
  --reasoning collaborative

# Or via API
curl -X POST http://localhost:8000/api/v1/research \
  -H "Content-Type: application/json" \
  -d '{"query": "transformer architectures", "reasoning_mode": "deep"}'
```

The agentic fabric is ready. The question is: what will you explore?

\begin{center} \small \textit{DOVA is open source and built on AWS Bedrock and the
Strands Agents SDK.}\ \textit{We welcome contributions at github.com/your-org/dova} \end{center}